

Projektuppgift

DT211G, Frontend-baserad webbutveckling

EuropaWiki

Uppbyggnad av webbplatsen

Emma Heikkinen



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Emma Heikkinen, emhe2504@student.miun.se

Utbildningsprogram: Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: 02, 2026

Sammanfattning

Denna rapport sammanfattar uppbyggnaden av Webbplatsen EuropaWiki. EuropaWiki är en webbplats som med hjälp av information från tre olika API presenterar fakta om länder i Europa. När användaren anger ett land i Europa som sökord hämtas information om det specifika landet och skrivs ut till skärmen.

Idén till webbplatsen kom från en förhoppning om att skapa något som skulle kunna vara användbart inför en resa. Webbplatsen är uppbyggd på engelska för att öka bredden av användare och ge möjlighet för resenärer världen över att kunna hitta information som kan vara bra att ha inför en resa, så som att veta valuta eller språk som används i landet.

Tanken med detta projekt är att arbetet ska sammanföra alla kursens uppgifter i en helhet. Det innefattar bland annat fetch-anrop (med async och await samt try/catch). Den inkluderar även Sass med flera filer för styling samt animationer för att ge mer liv till webbplatsen. På så sätt sammanförs kursens kunskaper till ett gemensamt projekt som nu kommer presenteras steg för steg i denna rapport.

Innehållsförteckning

Sammanfattning	iii
1 Inledning	1
1.1 Bakgrund och problemmotivering	1
2 Teori / Bakgrundsmaterial	2-3
2.1 Definition av termer och förkortningar	2-3
3 Metod	4
4 Konstruktion	5
5 Resultat	6
6 Slutsatser	7-8
Källförteckning	9
Bilaga A: Dokumentation	10

1 Inledning

Denna uppgift går ut på att skapa en webbplats eller webbapplikation med en tydlig målgrupp och grafisk profil, som får sitt innehåll från olika JSON-baserade webbtjänster, även kallat "API:er". Detta ska ske genom en så kallad "mashup" där informationen från dessa webbtjänster kombineras och integreras för att få fram en användbar webbplats med utökad information.

1.1 Bakgrund och problemmotivering

En webbplats ska skapas, där majoriteten av informationen på webbplatsen ska komma från minst två JSON-baserade webbtjänster. Webbplatsen ska vara användbar och API:erna som används bör vara öppna, men trots detta innehålla kvalitativ information. Användaren av webbplatsen ska kunna ha nytta av webbplatsen och den ska med fördel vara enkel att använda. Utöver detta ska webbplatsen ha en tydlig grafisk profil som tilltalar användaren samtidigt som utseendet på webbplatsen bör visa vad webbplatsen gör. Utifrån detta är följande steg avgörande i arbetsprocessen.

Först ut ska en idé skapas kring vad för typ av webbplats detta ska vara. Webbplatsen ska ha ett syfte som ger användaren något användbart. Det är viktigt att denna idé går att skapa med information från minst två olika API:er. Informationen från API:erna ska även stå för merparten av informationen på webbplatsen. Självklart ska även passande API:er tas fram.

När en idé på webbplats utformats ska en prototyp av webbplatsen skapas med en grundidé av webbplatsens utseende. Dock, här ska en del saker tas med i beräkningen. Vad har webbplatsen för målgrupp? Hur kan webbplatsens grafiska profil skapas för att korrelera med målgruppen för webbplatsen? Hur kan designen av webbplatsen öka användbarheten? Detta bör funderas över innan själva prototypen tar form.

När prototypen sedan är färdig ska denna börja utformas till en riktig webbplats. Detta arbete ska ske genom en automatiserad arbetsprocess (i detta fall används Vite) och med hjälp av de kunskaper som inhämtats under kursens gång. Med det i beaktning ska även Sass användas i designen av webbplatsen. Fetch-anrop (med `async` och `await` samt `try/catch`) ska tillämpas för att få fram informationen från API:erna. Arbetet ska dessutom versionshanteras och publiceras till GitHub.

Därefter ska en rapport skrivas, som steg för steg förklarar arbetsgången för hela projektuppgiften, från start till slut.

2 Teori

För att öka förståelsen av rapporten och dess innehåll presenteras här olika termer och begrepp som kan hjälpa läsaren med att begripa innehållet.

2.1 Definition av termer och förkortningar

JSON-baserad webbtjänst/API:

Ett API används för överföring av information mellan två platser, förslagsvis från ett system till ett annat. Ett exempel på detta kopplat till aktuellt projekt är överföring av information från RestCountries API till EuropaWiki. Fördelen med denna typ av överföring är att informationen kommer direkt från källan och uppdateras direkt hos källan, vilket minskar risken för att inkorrekt information publiceras på en webbplats. (1)(2)

Mashup:

Ett enda API innehåller i regel en viss typ av information. En mashup tillåter information från flera API:er att sammanföras för att öka bredden på information som presenteras där API:erna används, till exempel på en webbplats. I detta projekt sker detta på följande sätt:

1. Information om ett land hämtas från RestCountries API. Däribland landets huvudstad.
 2. Huvudstaden används sedan och skickas vidare till två andra API:er som tar fram ytterligare information om landets huvudstad.
 3. Detta presenteras som samlad information på webbplatsen EuropaWiki, även om informationen kommer i grunden kommer från tre olika källor.
- (2)(3)

Fetch-anrop (med async och await samt try/catch):

För att ta del av informationen från ett API behöver informationen hämtas. Därmed görs ett anrop till API:t. Async, await och try/catch gör hämtningen mer lättläst. Kort förklarat:

Async:

Deklarerar en async-funktion som alltid returnerar ett Promise, alltså ett objekt i form av ett löfte om att ett värde kommer att komma.

Await:

Pausar körningen av koden tills ett Promise är klart.

Try/catch:

Try kör koden, men fel kan uppstå och om de gör det körs catch som innehåller kod för att hantera felet. (4)(5)

Sass (CSS pre-processor):

Sedan tidigare har CSS använts för styling av webbplatser, men med större webbplatser leder det också till större CSS-filer som är svåra att underhålla och felsöka. Därför finns Sass, ett NPM-paket som är en expansion av CSS, där innehållet även är utökat med fler funktioner som till exempel nesting eller mixins. I Sass sker arbetet i flera olika filer, med olika syften för varje

fil, till exempel en fil för layout och en fil för detaljer. Alla filer har ändelsen .scss. När arbetet sedan är klart transpileras koden återigen till CSS då det är CSS som webbläsare begriper. Sass bidrar alltså till att arbetsgången, men styling, blir mycket simplare då det ger tillgång till flera funktioner och tydligare filer som är mindre och lättare att underhålla. (6)

Några funktioner inom Sass:

Variabler:

Används för lagring av information, till exempel typsnitt eller färger. Fördelen är att en variabel ändras på en plats, men den lagrade informationen ändras överallt där den används. Utvecklaren slipper alltså göra ändringar på flera ställen. (7)

Nesting:

Används för att kapsla selektorer inom varandra, vilket gör det enklare att strukturera och organisera styling utan att behöva skriva varje selektor separat. (7)

Mixins:

Används för att slippa onödiga upprepningar i din kod. Med mixins kan du återanvända CSS-deklarationer på flera platser utan att behöva skriva ut dem varje gång. (7)

Automatiserad arbetsprocess (Vite):

I projektets uppgiftsbeskrivning står det att en automatiserad arbetsprocess ska användas. Detta är, precis som det låter, en arbetsprocess där olika verktyg kan används för att automatisera delar av arbetet.

I arbetet med EuropaWiki används NPM-paketet Vite, ett modernare frontend verktyg, som förenklar och effektiviserar utvecklingen av webbapplikationer. Vite utgörs huvudsakligen av två delar, en utvecklingsserver och verktyg för att bygga ett projekt. Med hjälp av kommandon i terminalen kan utvecklaren enkelt starta ett projekt, se förändringar i realtid via en lokal utvecklingsserver och bygga en optimerad version av webbplatsen för produktion. Dessa kommandon körs ofta via scripts som finns definierade i en fil som heter package.json. (8)

3 Metod

För att skapa en grundidé om vilken webbplats som ska skapas kommer en webbläsare användas för att hitta olika API:er som finns tillgängliga att använda samt för att fundera ut vilka av dessa som hade passat att kombinera i detta projekt.

Därefter, när idén är klar, kommer Figma att användas för att skapa en designskiss av webbplatsen. Designskissen kommer ange webbplatsens grafiska profil, typsnitt, färger och uppbyggnad. Figma kommer även användas för att skapa en logotyp för webbplatsen. (9)

Fortsättningsvis, för utvecklingen, kommer flera olika verktyg att användas.

Utveckling och utvecklingsmiljö:

Utveckling och kodning kommer ske i Visual Studio Code, med Vite och Node.js command prompt för att automatisera arbetsprocessen. (10)(11)

Data från API:

För att hämta informationen från de olika API:erna kommer fetch, tillsammans med async och await samt try/catch, användas för att göra AJAX-anrop. (4)(5)

Styling:

För styling av webbplatsen kommer Sass att användas. Olika scss-filer kommer nyttjas för att skapa tydligare struktur, som exempel en fil för layout, en för variabler samt en för mixins. (6)(7)

Bilder, ikoner och bildkomprimering:

Bilder som behövs för webbplatsen kommer väljas ut från Pexels som har kostnadsfria, royaltyfria och upphovsrättsfria bilder. Dessa kommer redigeras och komprimeras i Photopea. Det är även i Photopea som ikoner kommer skapas. (12)(13)

Versionshantering:

För versionshantering kommer GitHub att användas för att göra commits och för att inte bara spara projektet lokalt. Genom GitHub kommer även olika grenar skapas, en "development"-gren där själva utvecklingsprocessen kommer ske samt en "main"-gren där själva publiceringen av webbplatsen kommer ske när utvecklingen är klar. (14)

Slutligen så kommer en rapport skrivas för att sammanfatta projektet. Denna kommer presentera arbetsgången som skett fram till den färdiga webbplatsen.

4 Konstruktion

Via en webbläsare gjordes efterforskningar om olika API:er för att undersöka om dessa är fria att använda samt för att se om de erbjuder användbar information. Av API-alternativen som fanns att tillgå skapades en idé för en informationswebbplats om Europa och för det ändamålet valdes tre passande API:er ut.

Därefter, när idén var klar påbörjades klurandet på ett passande utseende för en sådan webbplats. Målgruppen för en sådan webbplats kan variera både vad det gäller ålder och vilket land besökaren bor i, så designskissen skapades där utefter. Den grafiska profilen, med typsnitt och färgval baserades på tanken om ett lekfullt utseende, men med en tydlighet om vad det är för typ av webbplats. Samtidigt skapades designen med syftet att skapa en webbplats som är enkel att förstå hur den fungerar. Skissen skapades även på engelska för att öka bredden på webbplatsens användargrupp.

När designen började ta form valdes passande bilder ut till webbplatsen som sedan redigerades och komprimerades. I samband med detta skapades även en logotyp och en ikon för webbplatsen.

Sedermåra påbörjades själva utvecklingen med att kopiera och återanvända grundstrukturen från tidigare moment för att skapa en ny webbplats. Sedan skapades alla filer som behövdes för projektet, det vill säga HTML-filer, scss-filer samt JavaScript-filer. När detta var gjort publicerades projektet till GitHub där två grenar skapades, main samt development. (14)

När fil-strukturen var klar påbörjades arbetet i HTML-filerna för att bygga upp strukturen på webbplatsen likt den i designskissen. Detta innefattar bland annat sektioner, rubriker, textstycken och att lägga in alla bilder.

Fortsättningsvis användes Javascript för att hämta data från de API:er som valts ut (med hjälp av fetch med async och await, samt try/catch). Detta gjordes dock i omgångar på följande sätt:

1. Hämta informationen om länderna från RestCountries.
2. Med hjälp av huvudstadsinformationen från RestCountries hämtades tid- och temperaturinformationen från ytterligare två API:er (timeZone och OpenWeather). (2)(15)(16)

När detta var gjort påbörjades stylingen av webbplatsen med avsikt att efterlikna designskissen som skapats tidigare. En viktig del i stylingen av webbplatsen var att se till så den såg bra ut i olika enheter och likt hur det var tänkt enligt designskisserna (för de olika enheterna).

Avslutningsvis var det dags för de sista kontrollerna av webbplatsen. Det är viktigt att webbplatsen ser ut och fungerar som den ska, samt att testningen av webbplatsen ger bra resultat. När detta var klart publicerades webbplatsen till vald webbhost, varpå denna rapport också skrevs klart.

5 Resultat

Arbetet med den inledande planeringen och förarbetet resulterade i webbplatsen EuropaWiki. Detta är en webbplats där besökaren kan skriva in ett land i Europa i en sökruta och få tillbaka information om det landet. Informationen hämtas från tre olika API:er, där två av API:erna kräver information från det tredje API:et för att hämta sin information. Webbplatsen är på engelska för att nå en bredare målgrupp än endast svensktalande och avsedd för ett brett åldersspann av besökare från unga till äldre åldrar.

Efterforskningar efter passande API:er för projektet resulterade i tre olika API:er. Ett API för information om själva landet (RestCountries), ett API för information om tid och tidszon i huvudstaden (TimeZone) samt ett API för information om temperatur och väder i huvudstaden (OpenWeather).
(2)(14)(15)

Designskissen i Figma är färdig och innehåller det första utkastet av hur webbplatsen var tänkt att se ut. Den presenterar webbplatsens grafiska profil, typsnitt, färger och placering av bilder samt ikoner. Alltså den generella uppbyggnaden av webbplatsen, se länk (1.1), bilaga A. (9)

Utvecklingsprocessen har resulterat i en webbplats som är uppbyggd på två olika HTML-filer, en som presenterar själva startsidan där besökaren kan göra sina sökningar samt en "Om oss" sida som presenterar information om sidan och sidans skapare. Där finns även JSDocs att hitta. Utöver detta är strukturen för webbplatsens styling uppdelad i flera Sass filer, med olika syften för varje fil. Uppdelningen är fördelad enligt layout, utseende, animationer, detaljer, keyframes, mixins och variabler. JavaScript strukturen för webbplatsen är också uppdelad i flera filer med olika syften enligt en fil för vardera fetch, en fil som sköter sökningar samt en main-fil som initierar funktionerna när DOM är laddat. Det finns även två separata filer för JS-dokumentation.

Webbplatsen har publicerats till en vald webbhost och därefter genomgått flera tester för att undersöka om allt fungerar som det ska. Koden har validerats och utöver detta har bland annat laddningshastigheter och kontraster kontrollerats genom olika testverktyg. Därefter har länken till webbplatsen placerats i ReadMe-filen för projektet i Github tillsammans med länken till den här rapporten.

Projektarbetet är i och med detta färdigt och avslutat för nu. Det är nu inlämnat för rättning och feedback inväntas.

6 Slutsatser

För att sammanfatta projektet som varit så har det sammantaget gått bra.

Inledningen av arbetet gick till en början lite trögt då det var klurigt att komma på en idé till vilken webbplats som skulle skapas. Det jag visste var att jag ville komma på en egen idé, men då det finns så många olika API:er att välja mellan tog det tid att komma på vilka som skulle kunna kombineras på ett bra sätt. När jag sedan fann RestCountries API landade idén om att skapa en söktjänst för information om länder i Europa. Därefter valdes då två ytterligare API:er ut som passade in för det ändamålet.

När det sedan var dags att skapa en designskiss för webbplatsen gick det faktiskt väldigt bra. Jag testade mellan olika färgkombinationer, men ville välja en färg som kändes glad. Samtidigt behövde den passa ihop med färgerna i kartan i den ikon jag skapat precis innan. Jag tycker även att vissa delar av den färdiga webbplatsen efterliknar den ursprungliga planen jag hade i designskissen. Vissa förändringar behövde ske under arbetets gång och storlekar på typsnitt och bilder skiljer sig en del mellan prototypen och den färdiga webbplatsen. De storlekar som såg bra ut i designskissen såg inte alltid bra ut på webbplatsen och vice versa. Se bilder (1.3 och 1.4) nedan för jämförelser mellan prototyp och färdig webbplats, gällande likheter.



(1.3) designskiss



(1.4) webbplats

Något som gick mindre bra med den färdiga webbplatsen var responsiviteten. Jag hade en tanke om att sök-sektionen skulle kunna överlappa den översta bakgrundsbilden lite, men det blev inte alls bra när sidan skulle bli responsiv. Beroende på skärmstorlek hamnade rutan lite överallt och det såg helt enkelt inte bra ut. Därför gjorde jag valet att den i stället fick ligga under den bakgrundsbilden. Detta fungerar mycket bättre på alla enheter och det blir inte massor med felaktiga överlappningar. Här hade det dock varit bra om jag hade en till gren i GitHub för "testing", då dessa misstag uppdagades när projektet redan var merge:at till main. Detta resulterade i många korrigeringar och commits i main, vilket kunde ha undvikits med en "testing" gren. Detta tar jag med mig framöver. Se bild 1.5 och 1.6, bilaga A, för designkorrigeringar (ingen överlappning längre)).

En annan miss jag gjorde i början av arbetet med själva webbplatsen var att jag glömde publicera arbetet till GitHub och hann komma en ganska bra bit innan jag insåg detta. Därmed fick jag starta om ett nytt projekt och när jag publicerat detta till GitHub fick jag lägga in det arbete jag redan gjort och göra commits enligt de steg jag gjorde första gången jag skapade koden. Det var inte ett misstag som inte gick att rätta till, men det orsakade lite onödigt extraarbete.

Jag har funderat kring huruvida jag lyckats att skapa en webbplats som liknar en webbapplikation och att majoriteten av informationen ska vara hämtad från API:er. Jag anser att webbplatsen nog är lite mer lik en webbplats, dock presenteras det inte särskilt mycket information på startsidan som inte är hämtad från API:er. Det finns några förslag på resmål när besökaren kommer in på webbplatsen (se bild 1.7, bilaga A), men dessa försvinner när sökresultatet om ett land presenteras (se bild 1.8, bilaga A). Det är alltså placerade där för att skapa en inbjudande känsla på startsidan vid första anblick. Likaså för att ge en ökad förståelse kring webbplatsens syfte, det vill säga att presentera information om resmål i Europa. "Om oss" sidan är däremot den delen av webbplatsen som inte har någon information presenterad från API:er. Denna är dock egentligen ett tillägg utöver själva projektuppgiften, men jag anser att den tillför information till besökaren som kan behövas.

Vad det gäller animationer och effekter, som är en del av den här uppgiften, har jag försökt hitta en balans. Tanken är att webbplatsen ska få mer liv med de animationer och effekter jag valt ut, men samtidigt vill jag hålla det på en nivå där det inte blir för mycket eller där det upplevs rörigt. Exempel på animationer jag lagt till är rörelse på logotypen och färgändring på header-rubriken när sidan laddas. Jag har även lagt till flera hover-effekter på text och bilder.

Något annat som jag tycker blev bra är logotypen och ikonerna för webbplatsen. Ikonerna kan anses något enkel, men jag tycker att de båda återspeglar vad webbplatsen har för syfte (se bild 1.9 och 1.10 nedan). Det var även första gången jag skapade en logotyp i Figma (9).



(1.9)



(1.10)

Så för att sammanfatta och avrunda. Det har varit en väldigt rolig projektuppgift som sammantaget har gått bra att genomföra den. Slutresultatet liknar idén som togs fram i början av projektet. Jag anser att webbplatsen har ett bra syfte och att den kan vara användbar för många. Jag skulle i nuläget inte ändra något med webbplatsen och är förhållandevis nöjd med slutresultatet. Några små missar har skett längst vägen, men de har gått att åtgärda.

Källförteckning

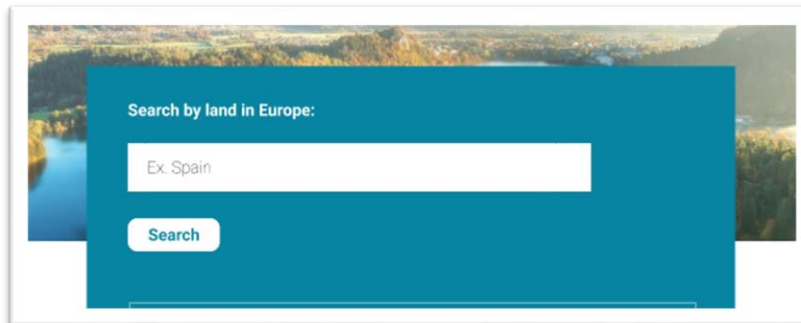
- [1] Skatteverket, "Vad är ett API?", <https://www.skatteverket.se/omoss/digitalasamarbeten/omvaraapier/vadaret-tapi.4.96cca41179bad4b1aaa4b8.html> Publicerad "u.å.". Hämtad 2026-03-19.
- [2] REST Countries, "Get information about countries via a RESTful API", <https://restcountries.com/> Publicerad "u.å.". Hämtad 2026-03-19.
- [3] IBM, "API Mashups", <https://www.ibm.com/docs/en/wams/wm-api-gateway-saas/11.1.0?topic=implementation-api-mashups> Publicerad 2026-01-19. Hämtad 2026-03-19.
- [4] Mdn, "async funktion", https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function Publicerad "2025-07-08". Hämtad 2026-03-19.
- [5] Mdn, "try...catch", <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch> Publicerad "2026-03-09". Hämtad 2026-03-19.
- [6] W3 Schools, "Sass Introduction", https://www.w3schools.com/sass/sass_intro.asp Publicerad "u.å.". Hämtad 2026-03-21.
- [7] Sass, "Sass basics", <https://sass-lang.com/guide/> Publicerad "u.å.". Hämtad 2026-03-21.
- [8] Vite, "Getting started", <https://vite.dev/guide/> Publicerad "u.å.". Hämtad 2026-03-21.
- [9] Figma, "Figma design" <https://www.figma.com/design/> Publicerad "u.å.". Hämtad 2026-03-21.
- [10] Visual Studio Code, "The open source AI code editor" <https://code.visualstudio.com/> Publicerad "u.å.". Hämtad 2026-03-21.
- [11] Node Js, "Run Javascript Everywhere" <https://nodejs.org/en> Publicerad "u.å.". Hämtad 2026-03-21.
- [12] Pexels, "De bästa kostnadsfria bilderna samt royaltyfria bilderna och videorna som delas av skapare" <https://www.pexels.com/sv-se/> Publicerad "u.å.". Hämtad 2026-03-21.

- [13] Photopea <https://www.photopea.com/> Publicerad "u.å.". Hämtad 2026-03-21.
- [14] GitHub, <https://github.com/> Publicerad "u.å.". Hämtad 2026-03-21.
- [15] Timezonedb, "Time Zone Database" <https://timezonedb.com/> Publicerad "u.å.". Hämtad 2026-03-22.
- [16] OpenWeather, "Smarter Weather for Everything you build" <https://openweathermap.org/> Publicerad "u.å.". Hämtad 2026-03-22

Bilaga A: Dokumentation

<https://www.figma.com/proto/qMOPJBVFYvxKgE51R2TodJ/Projekt-Dt211g?page-id=0%3A1&node-id=1-2&p=f&viewport=76%2C-1608%2C1&t=Tte6rtjZGnFiXAC8-1&scaling=min-zoom&content-scaling=fixed&starting-point-node-id=1%3A2&show-proto-sidebar=1> (1.1)

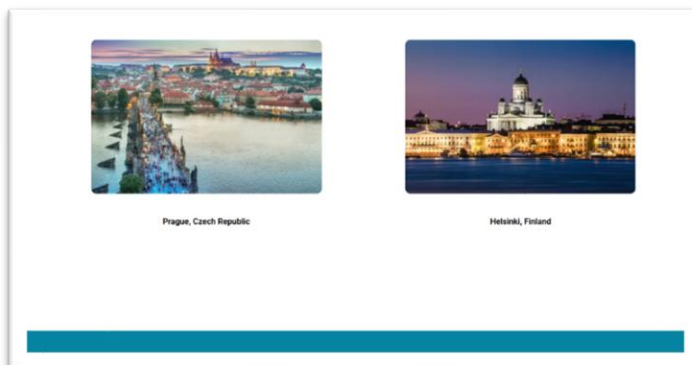
[Länk till webbsida](#) (1.2)



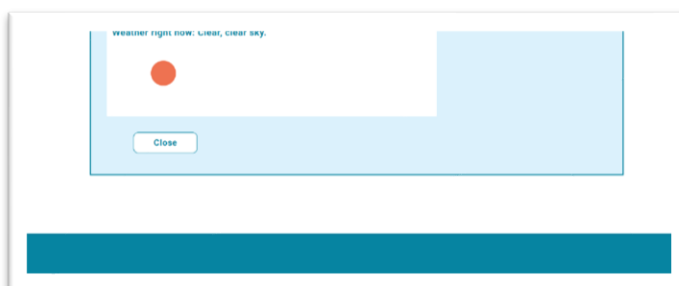
(1.5)



(1.6)



(1.7)



(1.8)